Stock Sentiment: Predicting Market Behavior from Tweets

David Cascão (20240851)[1], Elcano Gaspar (20241021)[1], Jorge Cordeiro (20240954)[1], and Rui Reis (20240854)[1]

[1]NOVA IMS, NOVA University of Lisbon, Portugal

May 2025

Abstract

This study addresses the challenge of sentiment classification in financial texts, a domain characterized by specialized terminology, concise expression, and nuanced sentiment indicators. We present a comprehensive evaluation of diverse methodologies spanning traditional machine learning, deep learning architectures, and generative AI models. Our systematic approach implements a six-stage pipeline encompassing exploratory data analysis, multi-variant preprocessing, feature engineering, model development, evaluation, and deployment. By comparing more than 45 distinct experimental configurations, we identify significant performance patterns across different modeling paradigms. Results demonstrate that while traditional models (SVM, LogisticRegression) with TF-IDF vectorization achieve competitive validation F1 scores (0.73), they exhibit substantial overfitting. Deep learning approaches (BiLSTM, MLP with GloVe embeddings) show improved generalization but moderate performance on minority sentiment classes. Transformer-based and generative AI models consistently outperform other approaches, with GPT-4o Mini achieving the highest macro F1 score (0.81) and precision (0.83). However, this performance comes at the cost of increased computational requirements, highlighting the critical trade-offs between prediction quality and efficiency. Our findings illuminate the evolution of financial sentiment analysis methodologies and provide empirical guidance for selecting optimal approaches based on specific application constraints. The research demonstrates that while large pre-trained language models excel at capturing domain-specific linguistic patterns, practical deployment considerations may favor more balanced solutions like SBERT with SVM classifiers when computational efficiency is prioritized.

Contents

1 Introduction

In this project, we address the challenge of predicting stock market sentiment from financial tweets, a task that combines natural language understanding with financial domain expertise. Our primary objective is to develop robust classification models that achieve optimal performance on this sentiment analysis task.

Despite this core objective, our chosen approach leads us through a deep technical exploration, and comprehensively compares a spectrum of Natural Language Processing (NLP) techniques, from traditional representative models to cutting-edge transformer architectures. This path not only serves to reinforce our classroom learning but also provides valuable insights into the evolution of text classification methodologies.

The project follows a systematic six-stage pipeline:

1. **Exploratory data analysis:** Examining dataset characteristics.
2. **Data preprocessing:** Cleaning and normalizing text data (tokenization, removing stop words, handling contractions, etc.).
3. **Feature engineering:** Implementing various text representation techniques (TF-IDF vectorization, Word2Vec embeddings, etc.) to capture financial sentiment nuances.
4. **Model development:** Implementing diverse classifiers.
5. **Comprehensive evaluation:** Rigorously assessing model performance.
6. **Final deployment:** Generating predictions for unseen data.

Through this structured approach, we not only fulfill the project requirements but also gain valuable insights into the practical considerations of implementing NLP solutions for financial text analysis.

The following sections detail our methodology, experiments, and findings at each stage of this process.

2 Data Exploration

The exploratory data analysis phase aimed to understand the structure, content, and challenges of the dataset to guide the design of preprocessing and modeling strategies. Given the classification objective, we conducted both quantitative and qualitative analysis.

2.1 Dataset overview

The training dataset comprises 7,637 labeled tweets, and the test set contains 3,276 unlabeled tweets. Each tweet consists of short texts with a financial context, including company tickers (e.g., `$TSLA`) and market-related expressions.

2.2 Class distribution

An initial assessment of the label frequencies revealed significant class imbalance, with approximately 65% of tweets classified as *Neutral*, 20% as *Bullish*, and 15% as *Bearish*. This imbalance poses a challenge for classification models, especially when using accuracy as the sole metric. To mitigate bias toward the dominant class, we later apply balancing techniques (e.g., SMOTE, class weights) and use the macro-averaged F1-score for fair evaluation.

2.3 Tweet length analysis

The histogram (Fig. 1) demonstrates a slight right-skewed distribution, with most tweets concentrated between 7–15 words, and a prominent peak around 10–12 words. This aligns with Twitter's historical character limitations and financial communication patterns where brevity is valued. The standard deviation of 4.65 words indicates moderate variability, with the dataset ranging from extremely brief 1-word tweets to more detailed 31-word messages.

Notably, many tweets follow a consistent format beginning with ticker symbols (like `$BYND`, `$CX`, `$ESS`) followed by brief analyst commentary or market updates. This length analysis helps inform our preprocessing strategy, particularly for transformer models with token limits, and suggests that financial sentiment can typically be conveyed in relatively compact messages.

2.4 Most Frequent Words

Word clouds were initially used during exploratory data analysis to visualize raw text characteristics, revealing the prevalence of terms like `https` and `MarketScreener` (see Fig. 2). Subsequent analysis, utilizing tokenized and cleaned text, allowed us to extract the most frequent unigrams and bigrams per class, with bar charts illustrating characteristic patterns for *Bullish*, *Bearish*, and *Neutral* sentiments.

2.5 Polarity scores

The boxplot visualization (Fig. 3) reveals a critical insight about TextBlob's sentiment analysis when applied to financial tweets: there is a distinct separation between *Bearish* and *Bullish* classifications, but a surprising similarity between *Bullish* and *Neutral* categories.

Bearish tweets display a distribution centered around zero with a substantial negative skew, clearly differentiating them from the other categories. However, both bullish and neutral tweets exhibit remarkably similar distributions with positive median values and comparable interquartile ranges.

This unexpected similarity suggests that TextBlob's general-purpose sentiment analyzer struggles to distinguish between positive and neutral sentiment in financial contexts, where industry-specific terminology and nuanced expressions might carry different connotations than in general text. The positive bias in supposedly neutral financial tweets indicates that standard NLP tools may misinterpret technical financial statements or objective market analyses as positive sentiment.

This finding highlights the importance of developing specialized sentiment analysis approaches for the financial domain that can better capture the subtleties between neutral reporting and genuinely positive market sentiment.

3 Data Preprocessing

Recognizing that different NLP models interpret input text through fundamentally different embedding strategies, we adopted a tiered preprocessing approach tailored to each model family.

For classical models such as Logistic Regression and SVM that rely on sparse count-based features (like Bag-of-Words or TF-IDF), we applied aggressive text normalization.

For neural networks like LSTMs we used moderate cleaning, preserving key linguistic patterns such as word order and lemmatization while removing noise. This allowed the embedding models to retain semantic relationships that enhance deep learning performance.

For transformer-based models like BERT, FinBERT, and GPT-4o, we applied minimal preprocessing, intentionally preserving the original structure, including case, hashtags, ticker symbols, and punctuation. These models are pre-trained on raw text and are most effective when fed data that resembles their training distribution.

By tailoring our preprocessing to each model's operational characteristics, we maximize their inherent strengths while maintaining comparability across approaches.

3.1 Cleaning for Traditional ML Models

The preprocessing of financial tweets presents unique challenges due to their specialized vocabulary, ticker symbols, and concise nature. To address these challenges and identify the optimal approach, we implemented a comprehensive text cleaning pipeline with multiple variant configurations.

3.1.1 Core Cleaning Methodology

Our preprocessing pipeline applies a sequential series of transformations to each tweet:

1. **Language detection** to identify non-English content
2. **Emoji conversion** to transform emoticons into textual descriptions
3. **Case normalization** by converting all text to lowercase
4. **HTML tag removal** to eliminate any markup elements
5. **Entity standardization** by replacing:
    - Monetary values with `MONETARY_VALUE_STAMP`
    - Date references with `DATE_STAMP`
    - Time references with `TIME_STAMP`
6. **Contraction expansion** (e.g., "don't" → "do not")
7. **URL removal** to eliminate web links
8. **Special character handling** to remove non-alphabetic characters
9. **Tokenization** to split text into individual tokens
10. **Stopword removal** to eliminate common words with limited semantic value
11. **Token normalization** through stemming or lemmatization

3.1.2 Preprocessing Variants

Rather than assuming a single optimal preprocessing approach, we systematically created five distinct preprocessing variants to empirically determine which combination of techniques yields the best performance for financial sentiment classification:

1. **Word tokenization with lemmatization** (`word_lemma`): Employs NLTK's `word_tokenize` function combined with WordNet lemmatization to normalize words to their dictionary forms, preserving semantic meaning.

2. **Tweet tokenization with Porter stemming** (`tweet_porter`): Uses Twitter-specific tokenization with Porter stemming, providing a moderate level of word normalization while maintaining sensitivity to Twitter-specific language patterns.

3. **Whitespace tokenization with Lancaster stemming** (`whitespace_lancaster`): Implements simple space-based tokenization with aggressive Lancaster stemming, maximizing term normalization at the potential cost of semantic precision.

4. **RegExp tokenization with Snowball stemming** (`regexp_snowball`): Applies regular expression pattern matching for tokenization with Snowball (Porter2) stemming, offering a balance between normalization strength and semantic preservation.

5. **Tweet tokenization without normalization** (`tweet_base`): Employs Twitter-specific tokenization without any stemming or lemmatization, preserving the original word forms while still removing noise.

Each variant was applied consistently across the training, validation, and test datasets, enabling a direct and fair comparison of model performance. This methodology allows us to isolate the impact of specific preprocessing strategies while maintaining consistency in other text-cleaning procedures. The visual impact of these preprocessing variants is illustrated in the word clouds shown in Fig. 4, highlighting how different techniques influence the lexical structure of the input data.

3.1.3 Implementation Approach

The preprocessing pipeline was implemented as a modular Python function that accepts configuration parameters to specify which techniques to apply. This design allows for systematic experimentation and reproducibility across different preprocessing strategies.

```
def clean(text_list, tokenizer_name='word_tokenize', lemmatize=False,
          stemmer_type=None, remove_html=True, convert_emoji=True,
          replace_monetary=True, replace_dates=True, replace_time=True,
          expand_contractions=True):
    # Implementation details omitted for brevity
```

3.2 Cleaning for Deep Learning Models

Deep learning models perform optimally with minimally processed text. This approach ensures the retention of critical contextual cues that might be lost with extensive preprocessing often required by traditional machine learning models. Our cleaning strategy for

these deep learning models was designed to be *lightly normalized* while preserving the *raw structure* of the text, stored in the `text_for_transformer` column.

This cleaning approach emphasized the following principles:

- **Minimal Alteration:** The text underwent light normalization, implying that transformations were kept to a minimum to maintain the natural form of the data.
- **Preservation of Contextual Elements:** Unlike traditional methods, this approach aimed to preserve elements such as punctuation, capitalization, emojis, hashtags, and ticker symbols (e.g., `$AAPL`, `#bullish`). These elements are vital for transformer models to accurately interpret sentiment and context.
- **Avoidance of Destructive Transformations:** Processes like stopword removal, stemming, or lemmatization, which can disrupt pre-trained token alignments and semantic relationships within the text, were avoided.

By adhering to these principles, the cleaning methodology maximized the models' ability to leverage their pre-trained knowledge and capture the nuances of the financial sentiment expressed in the tweets, ultimately contributing to enhanced model performance.

3.3 Cleaning for Generative/ Transformer Models

Similarly to deep learning models, transformers like BERT and GPT-4o work best with minimally processed text, unlike traditional models that need heavy preprocessing. They retain key contextual cues often lost through over-cleaning. Our cleaning approach for transformers was simple:

- Lowercasing only if required by the model tokenizer (e.g., BERT-cased vs uncased)
- Preserving punctuation, case, emojis, hashtags, and ticker symbols (e.g., `$AAPL`, `#bullish`)
- Avoiding stopword removal or stemming/lemmatization, which could disrupt pre-trained token alignment

This preserves the text's natural structure and maximizes model performance.

4 Feature Engineering

4.1 Approach for Traditional ML Models

After preprocessing the financial tweets through multiple variant approaches, we implemented a systematic feature engineering pipeline to transform the cleaned text into numerical representations suitable for machine learning algorithms. This stage is crucial for capturing the semantic nuances of financial sentiment while addressing dataset challenges.

4.1.1 Feature Representation Methodologies

Our feature engineering process generates three distinct types of text representations for each preprocessing variant:

1. **TF-IDF Vectorization** (Term Frequency-Inverse Document Frequency):
    - Transforms text into sparse matrices representing word importance
    - Configured with n-gram range (1,2) to capture phrases and term combinations
    - Limited to 5,000 maximum features to manage dimensionality
    - Applies minimum document frequency threshold to eliminate rare terms
    - Analyzes feature importance per sentiment class to identify class-indicative terms
2. **Word2Vec Document Embeddings**:
    - Trains domain-specific word embeddings on financial tweets
    - Creates 100-dimensional dense vector representations
    - Captures semantic relationships between financial terms
    - Generates document-level vectors by averaging constituent word vectors
    - Preserves semantic relationships (e.g., *bullish* relates to *upward*)
3. **Transformer-Based Contextual Embeddings**:
    - Leverages pre-trained Mini Sentence-BERT model (all-MiniLM-L6-v2)
    - Generates 384-dimensional fixed-length sentence embeddings
    - Captures contextual meaning and subtle sentiment indicators
    - Applies mean pooling over token embeddings for sentence representation
    - Standardizes feature distributions using z-score normalization

4.1.2 Class Imbalance Handling

As we know, we have class imbalance in our dataset. To address this challenge, we explored multiple mitigation strategies. However, due to time constraints, we were only able to implement and evaluate class weighting:

- **Class weighting** (selected approach): Assigns importance weights to classes inversely proportional to their frequency, ensuring balanced learning without data modification
- **Undersampling**: Reduces majority classes to match minority class size
- **Oversampling**: Increases minority classes through random duplication to match majority class size
- **SMOTE**: Generates synthetic examples for minority classes using nearest neighbor interpolation

4.1.3 Cross-Variant Feature Engineering

A key strength of our approach is the systematic generation of all feature types across every preprocessing variant, enabling empirical determination of optimal combinations:

```
preprocessing_versions = ['regexp_snowball', 'tweet_base', 'tweet_porter',
                          'whitespace_lancaster', 'word_lemma']

for version in preprocessing_versions:
    all_features[version] = fe.process_features(
        preprocessing_version=version,
        imbalance_strategy="weight"
    )
```

This results in 15 distinct feature sets (3 feature types $\times$ 5 preprocessing variants), enabling comprehensive experimentation to identify the most effective representation for financial sentiment classification.

4.1.4 Implementation Architecture

The feature engineering pipeline is implemented as a modular Python class with methods for:

- Loading preprocessed data from each variant
- Handling class imbalance through multiple strategies
- Generating each feature type with appropriate parameters
- Analyzing feature characteristics (vocabulary size, feature importance)
- Standardizing and persisting features for model development

This architecture enables systematic experimentation while maintaining reproducibility across different feature types and preprocessing approaches. The resulting feature sets provide a rich foundation for exploring traditional and advanced machine learning algorithms for financial sentiment classification.

4.2 Approach for Deep Learning Models

To capture the semantics of financial sentiment in tweets, we implemented two neural network architectures using pre-trained GloVe embeddings: a sequential BiLSTM model and a Multi-Layer Perceptron (MLP) model. These were chosen to compare the capabilities of recurrent and feedforward deep learning strategies under the same embedding representation.

Feature Representation Methodologies

Our deep learning feature engineering process comprises the following core components:

1. **Text Tokenization and Sequence Generation**

- We employed the Keras `Tokenizer` with a vocabulary limit of 20,000 words (`num_words=20000`) and an Out-Of-Vocabulary (OOV) token to manage unseen terms.
- The tokenizer was fitted on the training dataset (`train_trad['clean_text']`), converting each tweet into a sequence of integer word indices.
- Separate tokenized sequences were generated for the training, validation, and test datasets to avoid data leakage.

2. **Padding and Truncation**

   - The generated sequences were padded to a fixed length of 100 tokens (`maxlen=100`) using post-padding.
   - Tweets shorter than 100 tokens were padded with zeros, while longer tweets were truncated, preserving the initial content.

3. **Embedding Layer Initialization**

   - We utilized pre-trained GloVe embeddings (`glove.6B.100d`) to initialize the embedding layer. This external knowledge base captures rich semantic information learned from a large corpus.
   - An embedding matrix was constructed by mapping each word in our tokenizer vocabulary to its corresponding 100-dimensional GloVe vector.
   - Words not found in the GloVe dictionary were initialized with zero vectors, ensuring consistent dimensionality (`embedding_dim = 100`).

4. **Label Preparation**

   - Target sentiment labels were extracted from the `label` column of the training and validation sets and converted into NumPy arrays for direct model ingestion.

5 Classification Models

5.1 Traditional ML Models

After implementing multiple preprocessing variants and feature engineering approaches, we systematically evaluated different machine learning algorithms to identify the optimal combination for financial sentiment classification. Our model development pipeline follows a comprehensive experimental methodology designed to rigorously assess performance across all possible configurations.

5.1.1 Model Implementation Strategy

We implemented three traditional classification algorithms, each configured to address the specific challenges of financial sentiment analysis:

1. **Logistic Regression**:

   - Configured with L2 regularization (C=1.0) to prevent overfitting
   - Implemented with `balanced` class weights to compensate for class imbalance

- Used `lbfgs` solver for efficient optimization with multinomial classification

2. **Support Vector Machine (SVM)**:
   - Implemented with RBF kernel to capture non-linear decision boundaries
   - Utilized `scale` gamma parameter for appropriate feature scaling
   - Applied `balanced` class weights to handle imbalanced class distribution
   - Enabled probability estimation for ROC curve analysis
3. **K-Nearest Neighbors (KNN)**:
   - Configured with $k = 5$ neighbors for classification
   - Used cosine similarity metric to better capture semantic relationships
   - Adapted to handle both dense and sparse feature representations

5.1.2 Systematic Experimentation Framework

To identify the optimal approach, we implemented a grid search methodology across all possible combinations. This approach evaluated 45 distinct experimental configurations (3 models $\times$ 5 preprocessing variants $\times$ 3 feature types), enabling comprehensive performance assessment across the entire solution space.

5.1.3 Performance Evaluation Methodology

Each model was evaluated using multiple metrics to ensure robust assessment:

- **Accuracy**: Overall classification correctness
- **Macro F1 score**: Balanced measure across all sentiment classes
- **Overfitting analysis**: Difference between training and validation performance
- **ROC curves**: Class-specific discrimination capability
- **Confusion matrices**: Detailed error analysis by sentiment category

5.2 Deep Learning Models

5.2.1 BiLSTM + GloVe

- **Embedding Layer:** Initialized with 100-dimensional GloVe vectors, non-trainable.
- **Bidirectional LSTM:** 128 units with recurrent and input dropout (0.3), enabling learning from both past and future contexts.
- **Dense Layers:** ReLU-activated dense layer (64 units) with dropout (0.5), followed by a softmax output layer for classification.
- **Loss & Optimizer:** Trained using the Adam optimizer (`lr=0.0005`) and sparse categorical cross-entropy loss.

5.2.2 MLP + GloVe

- **Embedding Layer:** Identical GloVe initialization (non-trainable).
- **Flatten Layer:** Transforms sequences of embeddings into a flat vector suitable for dense layers.
- **Dense Layers:** Two fully connected ReLU layers with 256 and 128 units, respectively, and dropout rates of 0.4 and 0.3, followed by a softmax output layer.
- **Loss & Optimizer:** Same training configuration as the BiLSTM model.

Both models were trained using stratified 5-fold cross-validation, employing early stopping and best model checkpointing based on validation loss.

5.3 Transformer-Based Classifier Models

5.3.1 Sentence BERT

We worked on classifying tweets into three sentiment categories: **bullish**, **neutral**, or **bearish**. To do this, we used sentence embeddings from the SBERT model `all-mpnet-base-v2` and combined them with three traditional machine learning classifiers: Logistic Regression, Support Vector Machine (SVM), and Multi-layer Perceptron (MLP).

Approach:

- Tweets were cleaned by removing links, mentions, emojis, and special characters.
- Each tweet was transformed into a 768-dimensional vector using the `all-mpnet-base-v2` model.
- These embeddings were then used to train three classifiers:
  - Logistic Regression (baseline).
  - SVM with a linear kernel.
  - MLP for learning complex patterns.

5.3.2 Encoder-only LLM Models

BERTweet: It was used as a pre-trained language model trained on 850M+ English tweets, applied directly without fine-tuning. Despite its robustness in handling informal language from Twitter and StockTwits, its performance was lower than expected, particularly on sentiment extremes.

5.3.3 Fine-Tuned Encoder-Only LLM Models

FinBERT: It was fine-tuned using labeled financial sentiment data. Tokenization used FinBERT's pretrained tokenizer for compatibility. Training was conducted via the Hugging Face `Trainer` API with tracked evaluation metrics.

5.3.4 Generative Models and Ensemble Models

Prompt Engineering Strategy: We designed system prompts to simulate expert-level financial reasoning in a zero-shot setup, following best practices in clarity, brevity, and contextual specificity.

Zero-Shot Setup: We evaluated **GPT-4o** using Azure OpenAI for financial sentiment classification:

- No fine-tuning was performed.
- Integration was handled via the Azure SDK and API using `chat.completions.create()`.
- Constraints: high token cost and rate limits led to input optimization and batch processing.

Despite these limitations, GPT-4o proved a valuable benchmark against traditional and transformer-based approaches.

Ensemble Models: We implemented an ensemble approach using multiple open-source generative models: **OpenChat**, **Phi-3 Mini**, **Zephyr**, **Yi**, and **Starling**. All models were configured with detailed few-shot prompts to ensure consistent performance across the ensemble.

The main challenge encountered was the significant computational overhead of running multiple models locally, which resulted in extended processing times. Additionally, the individual models did not deliver extraordinary results, making the ensemble approach less compelling from a performance standpoint.

We experimented with two ensemble strategies:

- **Weighted Voting:** Better-performing models received higher weights in the final decision, with weights assigned based on individual model performance metrics.
- **Personalized Ensemble:** A custom strategy leveraging each model's strengths — OpenChat predictions for class 2 (neutral), Starling predictions for classes 0 and 1 (bearish/bullish), with majority voting used as a fallback for the remaining cases.

6 Final Results and Discussion

This section presents a comprehensive analysis of all models evaluated in our financial sentiment classification task. Through extensive experimentation with various combinations of preprocessing techniques, feature engineering approaches, and model architectures, we identified significant patterns in model performance and generalization capabilities.

Our experimental framework encompassed three distinct families of approaches: traditional machine learning models with engineered features, deep learning architectures, and generative AI models. Figure 5 illustrates the comparative performance of different feature engineering techniques across traditional models, revealing that TF-IDF vectorization consistently outperformed other approaches. However, as shown in Figure 6, these traditional models exhibited considerable overfitting.

The traditional model evaluation, as visualized in Figure 7, demonstrates that SVM generalization capabilities were compromised by significant overfitting. The SVM confusion

matrix (Figure 8) and ROC curves (Figure 9) further illustrate its strengths and limitations, particularly in distinguishing between bearish and bullish sentiments.

Our deep learning approaches showed mixed results. The BiLSTM architecture (Figure 11) demonstrated good generalization with training and validation curves converging over epochs, although it struggled with accurately classifying minority classes. In contrast, the MLP with GloVe embeddings (Figure 10) showed signs of overfitting, with validation loss increasing after initial improvements and a persistent gap between training and validation accuracy. Both models achieved modest performance on minority classes (bearish and bullish) while performing well on the majority neutral class, resulting in lower overall accuracy than our best-performing transformer-based models

6.1 Summary Table of All Models

Tables 1 and 2 provide a comprehensive overview of all evaluated models. Table 1 compares deep learning and generative AI approaches, while Table 2 presents the top-performing traditional machine learning configurations.

The summary metrics reveal several key insights:

1. **Performance Hierarchy:** Transformer-based models and generative AI consistently outperformed traditional approaches, with GPT-4o Mini achieving the highest macro F1 score (0.81) across all models.

2. **Preprocessing Impact:** For traditional models, the choice of preprocessing technique significantly influenced performance, with `regexp_snowball` and `tweet_base` configurations yielding the best results for Logistic Regression and SVM, respectively.

3. **Overfitting Patterns:** Traditional models exhibited substantial overfitting (0.14–0.26 difference between training and validation F1), while transformer-based approaches demonstrated better generalization.

4. **Precision–Recall Balance:** SBERT with an SVM classifier and OpenChat 3.5 with few-shot learning achieved the best recall metrics (0.84 and 0.83 respectively), while GPT-4o Mini delivered the highest precision (0.83).

6.2 Our Best Model

Our experimental results identify GPT-4o Mini as the best-performing model for financial sentiment classification, achieving an accuracy of 0.85 and a macro F1 score of 0.81. As shown in Figure 12, its confusion matrix demonstrates superior performance across all sentiment classes, with particularly strong results for neutral sentiment identification.

The GPT-4o Mini model offers several advantages over other approaches:

- **Superior Balance:** It achieves an optimal balance between precision (0.83) and recall (0.80), indicating reliable performance across all classes.

- **Minimal Preprocessing Requirements:** Unlike traditional models that require extensive preprocessing and feature engineering, GPT-4o Mini performs optimally with minimal text preprocessing.

- **Financial Domain Understanding:** The model effectively captures domain-specific financial language and sentiment nuances that traditional models struggle to identify.
- **Generalization Capability:** The model exhibits strong performance on validation data without the severe overfitting observed in traditional approaches.

When compared to the second-best model (SBERT transformer with SVM classifier), GPT-4o Mini delivers superior precision (0.83 vs. 0.76) with comparable accuracy (0.85 for both) and F1 score (0.81 vs. 0.79). This suggests that while both models effectively identify sentiment classes, GPT-4o Mini's predictions are more reliable, particularly for minority classes.

However, it's important to note that GPT-4o Mini is not the most computationally efficient solution. The model requires significantly more computational resources for both training and inference compared to traditional approaches and even smaller transformer models. This computational overhead creates important practical considerations for real-world deployment scenarios, especially in applications requiring low-latency predictions or deployment on resource-constrained environments. The performance advantages of GPT-4o Mini highlight the value of large pre-trained language models for specialized financial text analysis tasks, where capturing subtle linguistic patterns and domain-specific terminology is critical for accurate sentiment classification. Yet these results also emphasize the critical trade-offs between model complexity, computational requirements, and performance metrics that must be considered when selecting an optimal approach for financial sentiment analysis. In scenarios where computational efficiency is prioritized over marginal performance gains, the SBERT+SVM configuration may represent a more balanced solution.

7 Conclusion

Our study provides a comprehensive evaluation of financial sentiment classification approaches, revealing a clear performance hierarchy: generative AI models (notably GPT-4o Mini) outperform transformer-based architectures, which surpass deep learning models and traditional machine learning techniques. This performance advantage, however, comes with substantially higher computational costs.

We demonstrate that minimal preprocessing combined with advanced language models effectively captures financial domain nuances, while extensive feature engineering yields diminishing returns compared to the inherent capabilities of pre-trained models.

Future research should explore:

1. Multimodal integration with numerical financial data.
2. Cross-lingual extension for global market analysis.

This research provides a foundation for selecting appropriate financial sentiment classification methodologies, balancing performance requirements against computational constraints in this increasingly important domain.

Appendices

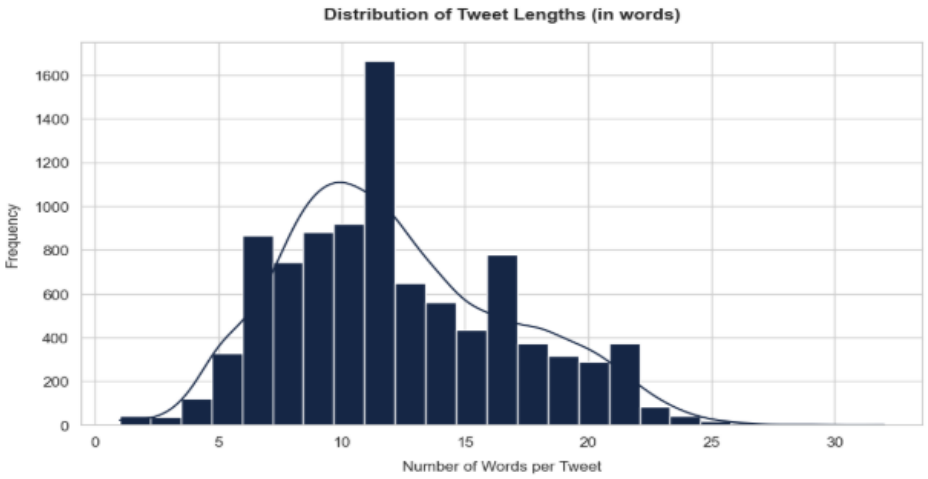

Figure 1: Distribution of Tweet Lengths

Figure 2: Word cloud of raw tweets

Model	Acc	F1	Prec	Rec	Notes
GPT-4o Mini	0.85	0.81	0.83	0.80	
SBERT transformer + MLP classifier	0.84	0.79	0.78	0.79	
SBERT transformer + SVM classifier	0.85	0.79	0.76	0.84	
Fined-tuned FinBert	0.85	0.79	0.78	0.80	
openchat 3.5	0.81	0.78	0.75	0.83	few shot 2h, good on bullish
openchat 3.5	0.78	0.76	0.73	0.83	few shot
openchat 3.5	0.79	0.76	0.73	0.80	zero shot 40
SBERT transformer + Logistic classifier	0.82	0.76	0.73	0.80	
Logical Personalized Ensemble	0.79	0.75	0.73	0.78	
Weighted Ensemble Voting	0.78	0.75	0.73	0.78	
openchat 3.5	0.77	0.75	0.72	0.84	zero shot
Phi-3 Mini	0.71	0.69	0.67	0.77	few shot 15min, good on bearish
BiLSTM	0.78	0.67	0.74	0.63	
Phi-3 Mini	0.71	0.65	0.66	0.65	zero shot 15min
MLP	0.72	0.59	0.63	0.57	

Table 1: Comparison of Deep learning and Gen AI Models in test set, sorted by F1 score

Model	Preprocessing	Feature	Train Acc	Train F1	Val Acc	Val F1	Test F1	Overfit
LogisticRegression	regexp_snowball	tfidf	0.89	0.87	0.79	0.73	0.72	0.14
SVM	tweet_base	tfidf	0.98	0.97	0.81	0.73	0.71	0.25
LogisticRegression	word_lemma	tfidf	0.89	0.87	0.79	0.73	0.71	0.14
LogisticRegression	tweet_porter	tfidf	0.90	0.87	0.79	0.73	0.70	0.15
SVM	word_lemma	tfidf	0.98	0.97	0.81	0.72	0.70	0.25
SVM	regexp_snowball	tfidf	0.98	0.97	0.80	0.72	0.69	0.25
SVM	whitespace_lancaster	tfidf	0.98	0.97	0.80	0.72	0.69	0.26
SVM	tweet_porter	tfidf	0.98	0.97	0.80	0.72	0.69	0.26
LogisticRegression	whitespace_lancaster	tfidf	0.89	0.87	0.77	0.71	0.68	0.15
LogisticRegression	tweet_base	tfidf	0.90	0.87	0.77	0.70	0.67	0.17

Table 2: Top 10 combinations across all traditional models sorted by test F1 score

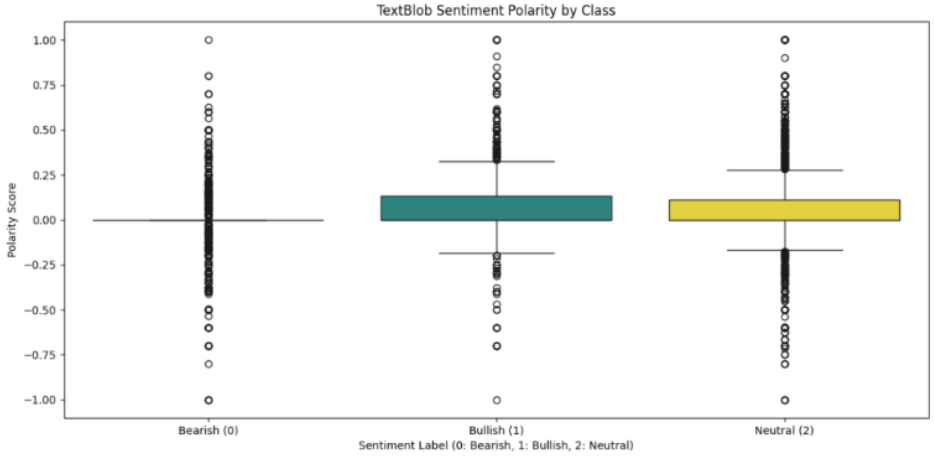

Figure 3: TextBlob Sentiment Polarity by Class

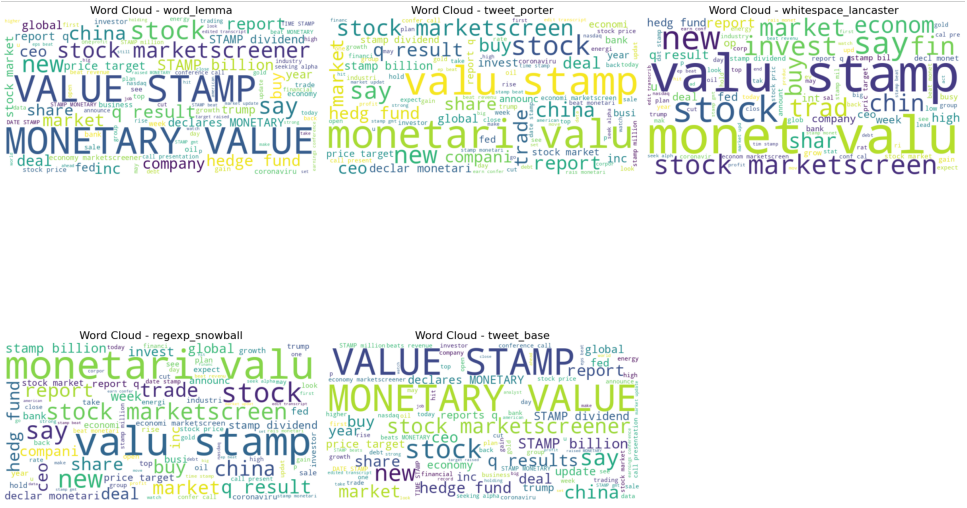

Figure 4: Word clouds after preprocessing

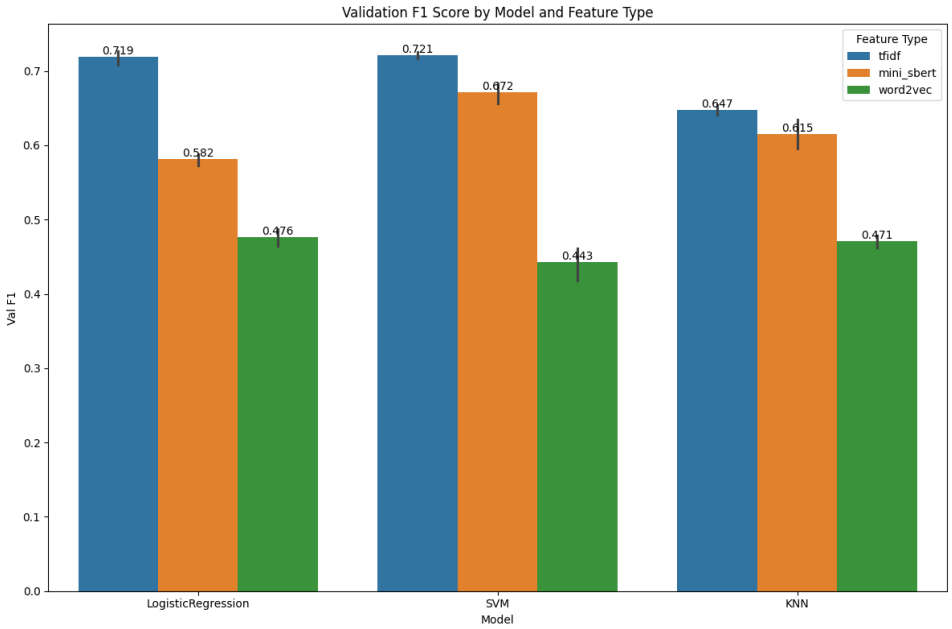

Figure 5: Comparision of feature engineering techniques in traditional models in terms of performance

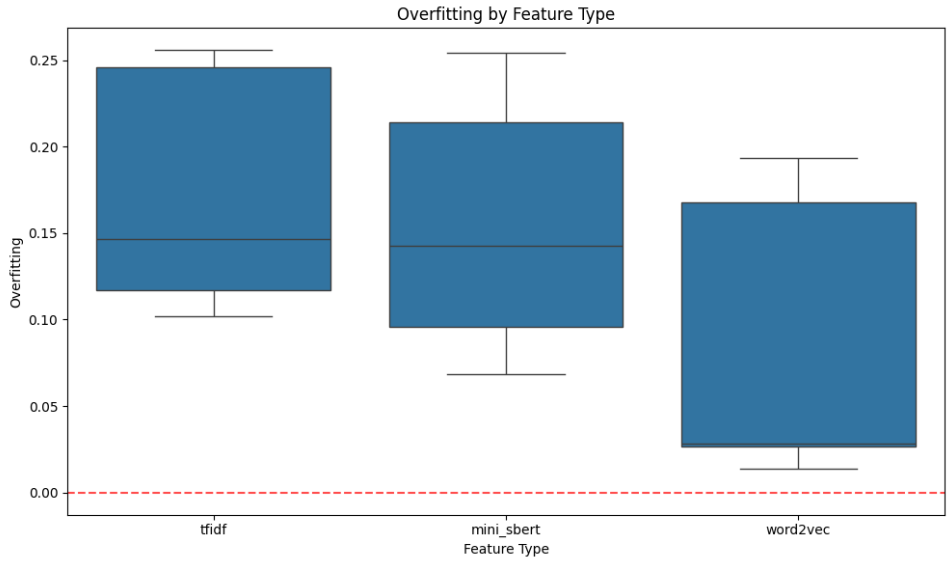

Figure 6: Comparision of feature engineering techniques in traditional models in terms of overfitting

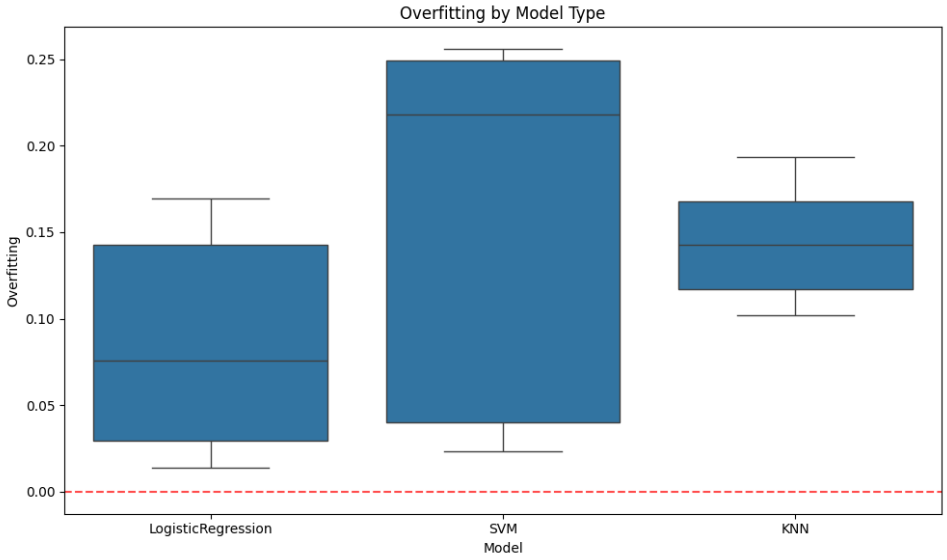

Figure 7: Comparision of traditional models in terms of overfitting

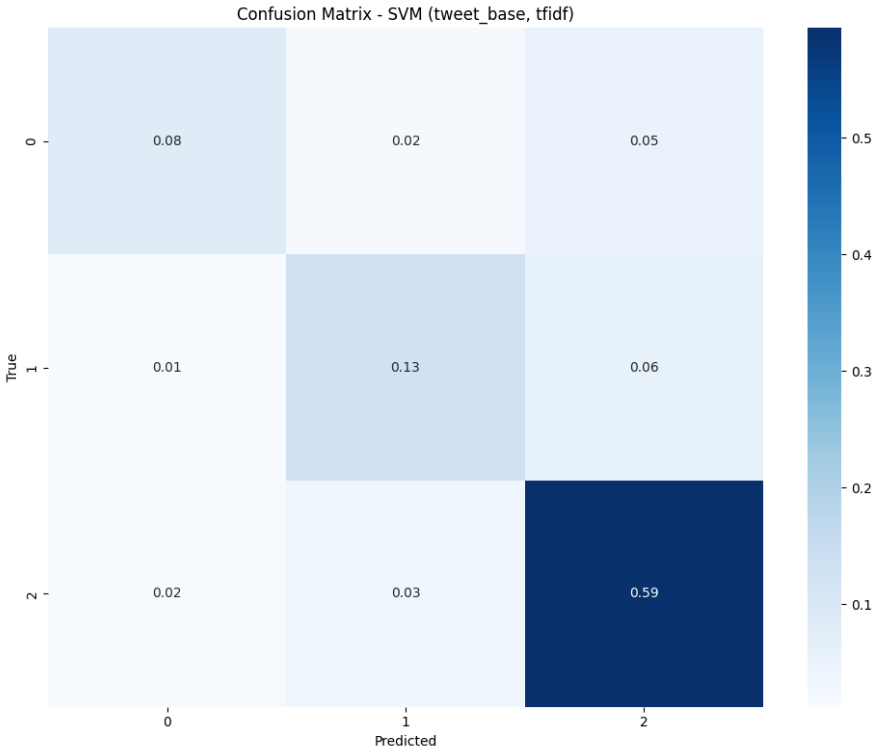

Figure 8: Confusion matrix of SVM

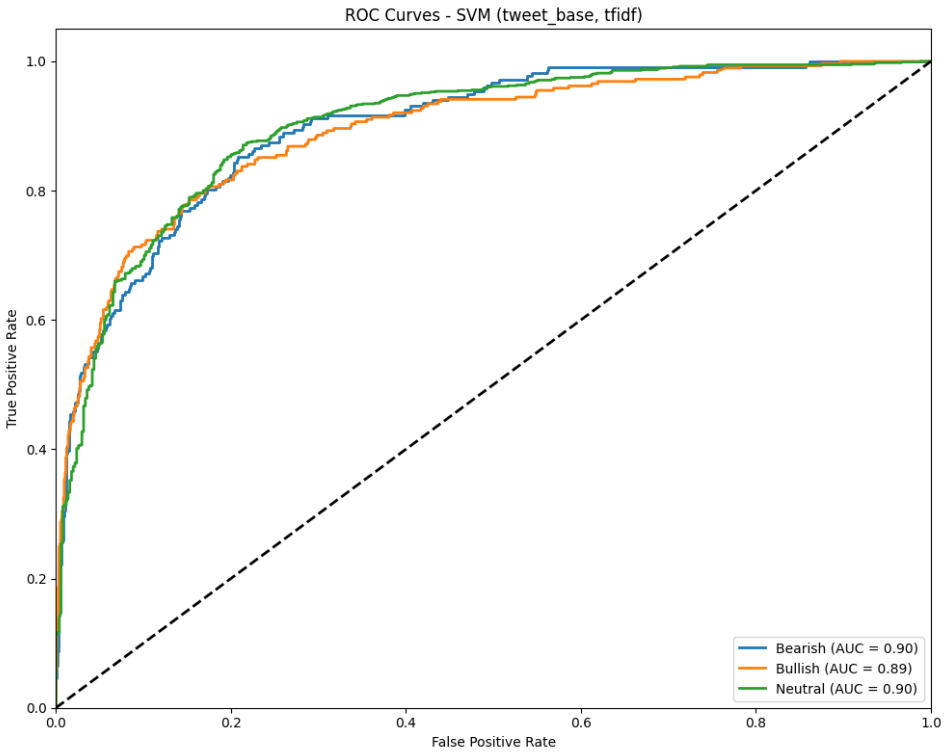

Figure 9: Roc curve of svm

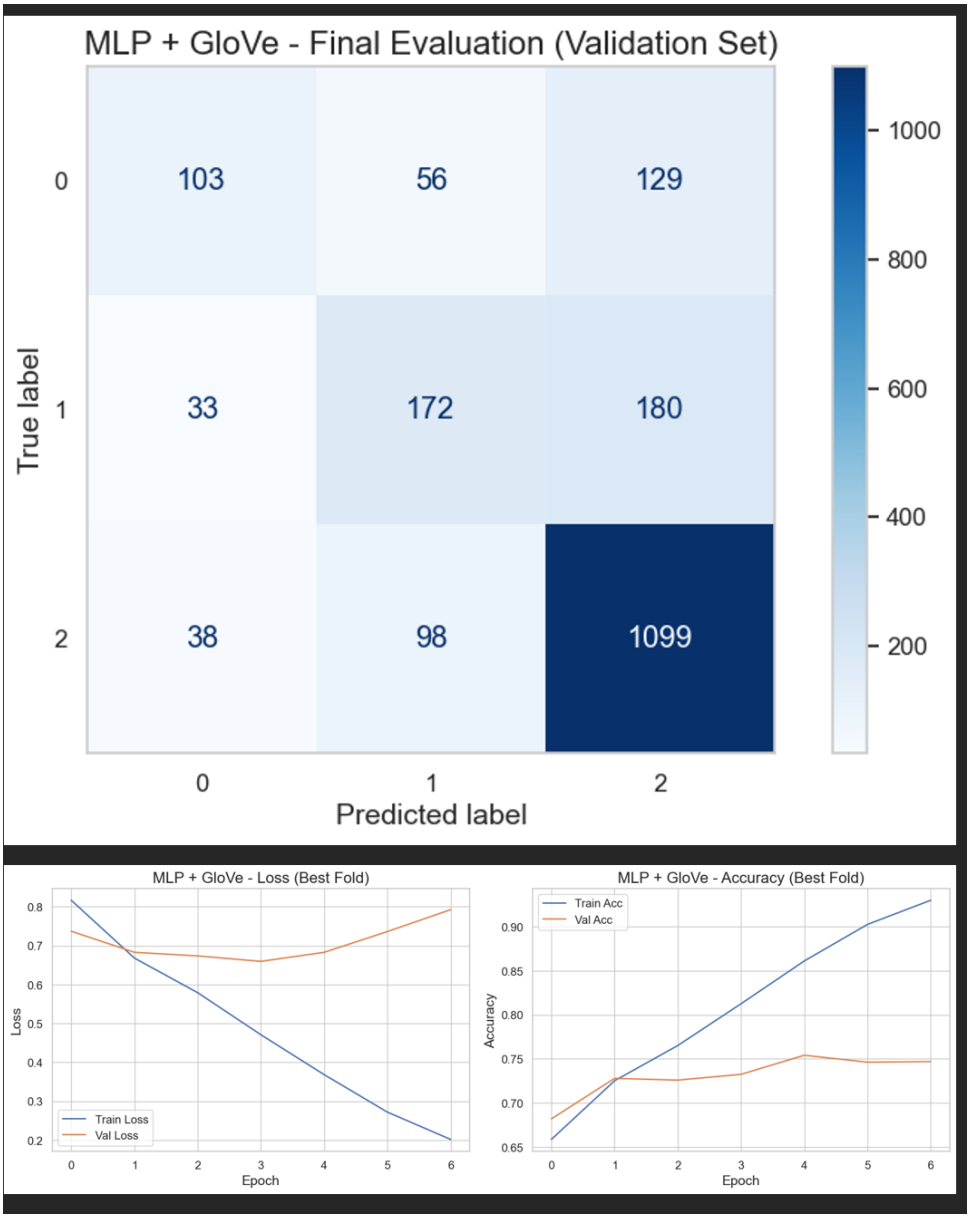

Figure 10: MLP with GloVe embeddings: Confusion matrix on validation set (top) and training curves showing loss and accuracy over epochs (bottom)

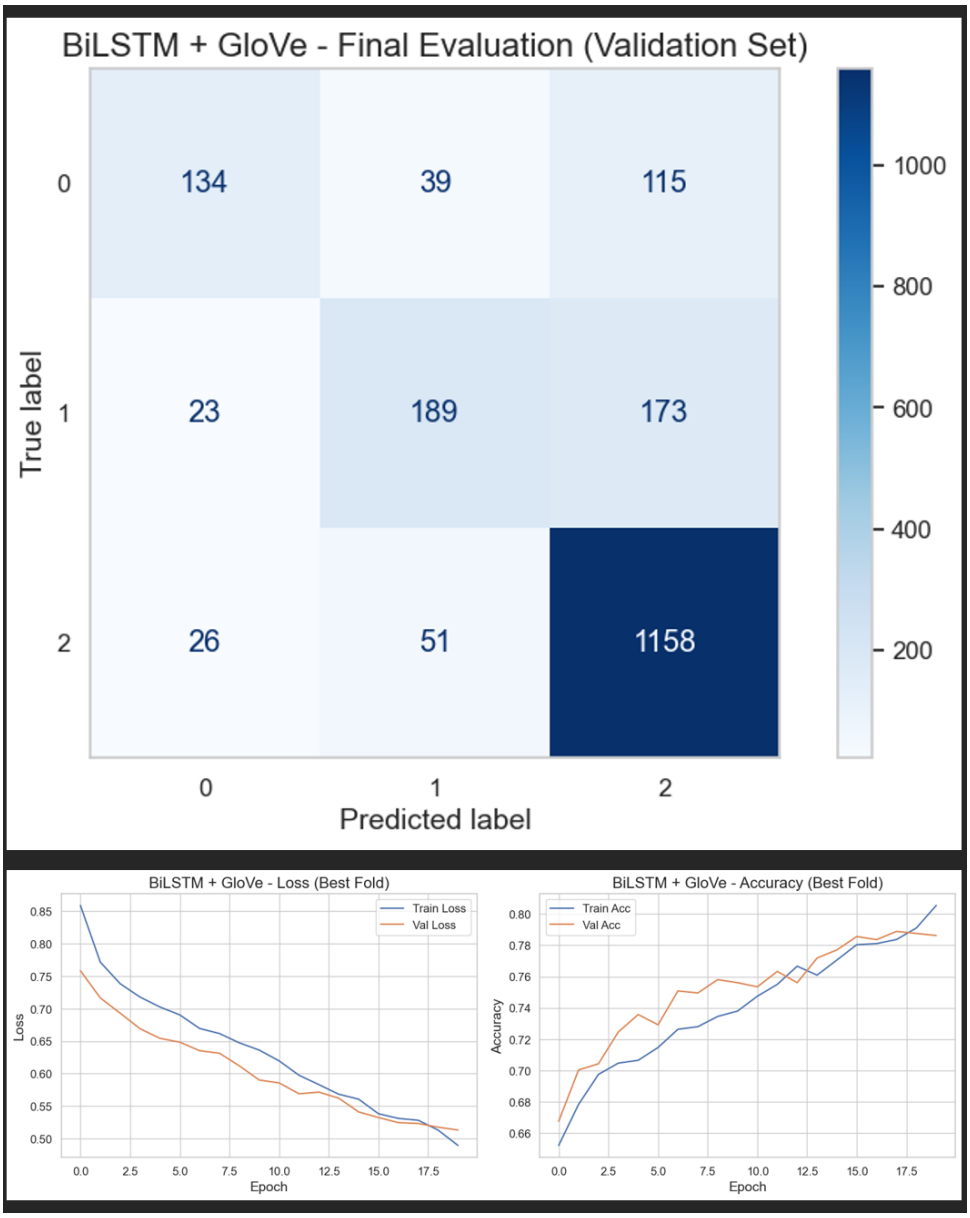

Figure 11: BiLSTM: Confusion matrix on validation set (top) and training curves showing loss and accuracy over epochs (bottom)

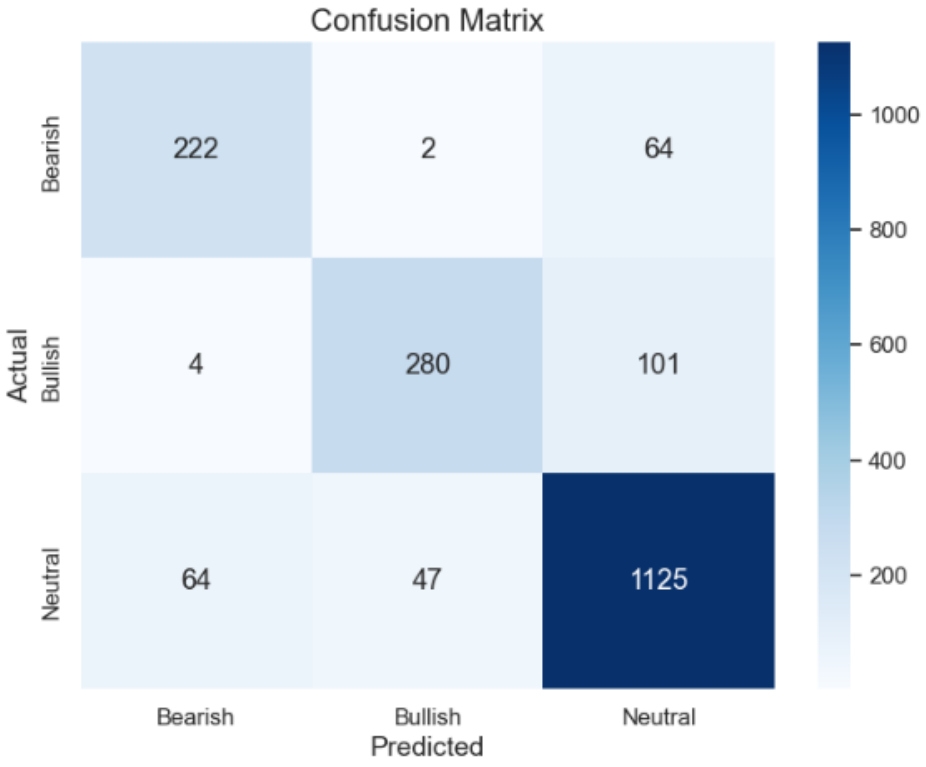

Figure 12: Confusion Matrix GPT-4o mini

References

[1] Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv:1908.10063*. https://arxiv.org/abs/1908.10063

[2] Bartov, E., Faurel, L., & Mohanram, P. S. (2018). Can Twitter Help Predict Firm-Level Earnings and Stock Returns? *The Accounting Review*, 93(3), 25–57. https://doi.org/10.2308/accr-51865

[3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. https://aclanthology.org/N19-1423/